作业 _Week13

20244227004 蔡博凯

2024 年 12 月 2 日

第一题

请以机器翻译为例，写出Transformer工作的伪代码。

解答

1. 介绍

伪代码实现了一个基于 Transformer 架构的模型，主要用于机器翻译任务。模型包括编码器（Encoder）和解码器（Decoder）两大部分。每个部分由多个层堆叠而成，每层包含自注意力机制（Self-Attention）、跨注意力机制（Decoder 特有）和前馈神经网络（Feed-Forward Network）。

2. 模型通过以下步骤工作：

- **初始化**：包括嵌入层、位置编码、编码器和解码器层的创建，以及输出层的初始化。
- **编码器前向传播**：将输入序列嵌入并加上位置编码，通过每层的自注意力机制和前馈网络提取特征。
- **解码器前向传播**：通过目标序列和编码器输出，逐步生成目标序列。解码器层包括自注意力、跨注意力和前馈网络。

- **翻译过程**：首先编码源语言。初始化目标序列并自回归生成目标语言，直到生成结束符号 <EOS>。

3. 详细代码请参考 transformer.py.

第二题

Transformer中使用了全局位置编码（PE），PE通常可以直接加在输入端上，请问为什么这样做不影响输入本身特征的表达（可从注意力权重计算的角度出发解释）。

解答

解释如下

假设输入序列的嵌入矩阵为 $X \in \mathbb{R}^{n\times d}$，其中 n 是序列长度，d 是每个词的嵌入维度。位置编码矩阵为 $P \in \mathbb{R}^{n\times d}$，其维度与输入嵌入矩阵相同。位置编码的加入通过加法操作进行：

$$X' = X + P$$

接着，我们计算 Query、Key 和 Value 向量，分别为：

$$Q = X'W_Q = (X+P)W_Q = XW_Q + PW_Q$$

$$K = X'W_K = (X+P)W_K = XW_K + PW_K$$

$$V = X'W_V = (X+P)W_V = XW_V + PW_V$$

其中 $W_Q \in \mathbb{R}^{d\times d_k}$、$W_K \in \mathbb{R}^{d\times d_k}$、$W_V \in \mathbb{R}^{d\times d_v}$ 是可学习的权重矩阵，d_k 和 d_v 分别是 Query、Key、Value 向量的维度。在自注意力机制中，我们通过计算 Query 和 Key 之间的点积相似度来得到注意力权重：

$$QK^T = (XW_Q + PW_Q)(XW_K + PW_K)^T$$

展开后：

$$QK^T = (XW_Q)(XW_K)^T + (XW_Q)(PW_K)^T + (PW_Q)(XW_K)^T + (PW_Q)(PW_K)^T$$

最终得到：

$$QK^T = XW_QX^TW_K + XW_QP^TW_K + PW_QX^TW_K + PW_QP^TW_K$$

其中第一项 $XW_QX^TW_K$ 是原始输入后三项则是位置编码 P 与输入之间的交互项。

接下来，我们通过 Softmax 计算注意力权重矩阵 A：

$$A = \text{Softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)$$

最后，输出是基于注意力权重对 Value 向量的加权求和：

$$O = AV = A(XW_V + PW_V)$$

展开后：

$$O = A(XW_V) + A(PW_V)$$

通过位置编码的加法操作，输入向量的语义表达保持不变，而位置编码则影响了注意力计算的过程。位置编码通过与输入嵌入的加法操作，影响了注意力权重的计算和最终的输出，但不会改变输入本身的语义信息。

总结

由于位置编码是加性而非乘性地加入到输入向量中，它并不改变原始输入向量本身的特征表示。具体来说，输入向量的语义内容（如单词的含义）和位置编码（如位置的顺序关系）是分开的。虽然位置编码影响了注意力权重的计算，但它不会改变每个单词的原始特征表示。

A Transformer 伪代码

```
class TransformerModel:
    # 初始化 Transformer 模型的组件
    def __init__(self, num_layers, d_model, num_heads, d_ff,
        vocab_size, max_seq_len):
```

```
4           # 初始化嵌入层和位置编码
5           self.embedding = initialize_embedding_matrix(vocab_size,
                d_model)  # 嵌入矩阵：将输入的单词索引映射到稠密向量
                空间
6           self.position_encoding = positional_encoding_matrix(
                max_seq_len, d_model)  # 位置编码：为每个单词添加位置
                相关的向量
7
8           # 创建编码器层，重复 num_layers 次，每一层包含自注意力机
                制和前馈网络
9           self.encoder_layers = [
10              self._create_encoder_layer(d_model, num_heads, d_ff)
11              for _ in range(num_layers)
12          ]
13
14          # 创建解码器层，重复 num_layers 次，每一层包含自注意力机
                制、跨注意力机制和前馈网络
15          self.decoder_layers = [
16              self._create_decoder_layer(d_model, num_heads, d_ff)
17              for _ in range(num_layers)
18          ]
19
20          # 输出层：通过一个线性变换将 decoder 输出映射到词汇表大
                小的维度
21          self.output_layer = initialize_linear_layer(d_model,
                vocab_size)
22
23      # 创建编码器层，包含自注意力机制和前馈网络
24      def _create_encoder_layer(self, d_model, num_heads, d_ff):
25          return (
26              create_multihead_attention(d_model, num_heads),  #
                    创建多头自注意力机制
```

```
27              create_feed_forward_network(d_model, d_ff)       #
                    创建前馈神经网络
28          )
29
30      # 创建解码器层，包含自注意力机制、跨注意力机制和前馈网络
31      def _create_decoder_layer(self, d_model, num_heads, d_ff):
32          return (
33              create_multihead_attention(d_model, num_heads),  #
                    自注意力机制
34              create_multihead_attention(d_model, num_heads),  #
                    跨注意力机制，用于将解码器与编码器连接
35              create_feed_forward_network(d_model, d_ff)       #
                    前馈神经网络
36          )
37
38      # 编码器的前向传播
39      def encoder(self, input_sequence):
40          # 将输入序列嵌入并加上位置编码
41          embedded_input = (
42              self.embedding[input_sequence] +  # 输入的词嵌入
43              self.position_encoding[:len(input_sequence)]  # 位置
                    编码
44          )
45
46          # 逐层通过编码器
47          for self_attention, feed_forward in self.encoder_layers:
48              embedded_input, _ = self_attention(embedded_input,
                        embedded_input, embedded_input)  # 自注意力机制
49              embedded_input = feed_forward(embedded_input)  # 前
                    馈网络
50
51          # 返回编码器的输出
52          return embedded_input
```

```
53
54      # 解码器的前向传播
55      def decoder(self, target_sequence, encoder_output):
56          # 将目标序列嵌入并加上位置编码
57          embedded_target = (
58              self.embedding[target_sequence] +  # 目标序列的词嵌入
59              self.position_encoding[:len(target_sequence)]  # 位置编码
60          )
61
62          # 逐层通过解码器
63          for self_attention, cross_attention, feed_forward in self.decoder_layers:
64              # 自注意力机制：目标序列自身的注意力
65              embedded_target, _ = self_attention(embedded_target, embedded_target, embedded_target)
66              # 跨注意力机制：解码器与编码器输出之间的注意力
67              embedded_target, _ = cross_attention(embedded_target, encoder_output, encoder_output)
68              # 前馈网络
69              embedded_target = feed_forward(embedded_target)
70
71          # 返回解码器的输出
72          return embedded_target
73
74      # 翻译过程
75      def translate(self, source_sequence):
76          # 1. 编码源语言句子
77          encoder_output = self.encoder(source_sequence)
78
79          # 2. 初始化目标序列（通常从 <START> token 开始）
80          target_sequence = [start_token]
```

```
81
82          # 3. 自回归地生成目标语言句子
83          while True:
84              # 3.1 解码目标序列
85              decoder_output = self.decoder(target_sequence,
                    encoder_output)
86              # 3.2 通过输出层生成预测
87              output_logits = softmax(self.output_layer(
                    decoder_output), axis=-1)
88              # 3.3 选择概率最高的词作为下一个 token
89              next_token = argmax(output_logits)
90              target_sequence.append(next_token)  # 将生成的 token
                     添加到目标序列中
91
92              # 3.4 判断是否生成结束符号 <EOS>
93              if next_token == eos_token:
94                  break
95
96          # 4. 返回生成的目标序列
97          return target_sequence
```